

New practice questions — same shapes as the 2026 paper, answers at the end of each

None of these are real exam questions: they are the 2026 exercises re-skinned so you can run the whole recipe once more without copying. Time yourself: **unit 15 min · REST 20 min · black-box 8 min · deployment 10 min · quiz 10 min.**

P1 — Unit testing: `DataPlan` (shape of `CloudServer`)

A mobile `DataPlan` has `balance` (double, GB), a home `network` (enum `Network`: `EDGE`, `LTE`, `FIVE_G`, `WIFI`) and an `optimizationLevel` (int).

`useData(Network network, double mb)` — cost is `mb * 2.0`; if `network` equals the plan's home network, cost is `mb * 0.5`. If `balance < cost`: return `new UsageResult(false, 0.0, 0.0)` and change nothing. Otherwise deduct the cost and return `new UsageResult(true, speed, cost)` where `speed = (optimizationLevel * 50.0) / (mb + 5.0)`, **plus 20.0** if the network matches.

`topUp(double amount)` — if `amount <= 0.0` return the balance unchanged. Add `amount * (1.1 + optimizationLevel * 0.02)`, but never exceed `500.0 + optimizationLevel * 25.0`. Return the new balance.

Write: `testUseDataHomeNetwork`, `testUseDataOtherNetwork`, `testUseDataInsufficientBalance`, `testTopUpNormal`, `testTopUpCap`, and one test for the exact-fit boundary.

Answer (balance 100, `LTE`, level 2 unless stated)

Test	Setup → call	Assert
home network	<code>useData(LTE, 10)</code>	success · cost 5.0 · balance 95.0 · speed $100/15 + 20 = \mathbf{26.6667}$
other network	<code>useData(WIFI, 10)</code>	success · cost 20.0 · balance 80.0 · speed 6.6667
insufficient	balance 10 , <code>useData(WIFI, 10)</code> (needs 20)	<code>assertFalse</code> · balance still 10.0 · speed 0.0 · cost 0.0
exact fit	balance 5 , <code>useData(LTE, 10)</code> (needs exactly 5)	success · balance 0.0
top-up normal	balance 10, <code>topUp(10)</code>	$10 + 10 \times 1.14 = \mathbf{21.4}$, returned and stored
top-up cap	balance 10, <code>topUp(60000)</code>	550.0 ($= 500 + 2 \times 25$), not a remainder
non-positive	<code>topUp(0)</code> , <code>topUp(-5)</code>	unchanged

```

@Test void testUseDataHomeNetwork() {
    DataPlan plan = new DataPlan(100.0, Network.LTE, 2);
    UsageResult r = plan.useData(Network.LTE, 10.0);           // capture the result!
    assertTrue(r.isSuccess());
    assertEquals(10.0 * 0.5, r.getCost(), 1e-9);
    assertEquals(100.0 - 10.0 * 0.5, plan.getBalance(), 1e-9); // what is LEFT, not the
cost
    assertEquals((2 * 50.0) / (10.0 + 5.0) + 20.0, r.getSpeed(), 1e-9); // 50.0 not 50: no
int division
}

```

P2 — REST: LoanResource (shape of Ticket Manager)

Resource /loans with Loan {Long id, String bookTitle, String borrower, boolean returned}. POST /loans 200, or 400 if the body has an id · GET /loans 200 · GET /loans/{loanId} 200 / 404 · PUT /loans/{loanId} 200 / 400 (ids differ) / 404 · DELETE /loans/{loanId} 204 always. **Extension:** GET /loans takes an optional parameter onlyOpen (default false); when true, return only loans that are not returned. **Extension 2:** PUT on a loan that is already returned answers **409 Conflict**.

Answer

```

@RestController @RequestMapping("/loans")
public class LoanResource {
    private final LoanService loanService;
    public LoanResource(LoanService loanService) { this.loanService = loanService; }

    @PostMapping
    public ResponseEntity<Loan> create(@RequestBody Loan loan) {
        if (loan.getId() != null) throw new RuntimeException(HttpStatus.BAD_REQUEST);
        return ResponseEntity.ok(loanService.saveLoan(loan));
    }

    @GetMapping
    public ResponseEntity<List<Loan>> getAll(@RequestParam(defaultValue = "false") boolean
onlyOpen) {
        return ResponseEntity.ok(loanService.getAllLoans(onlyOpen));
    }

    @GetMapping("/{loanId}")
    public ResponseEntity<Loan> get(@PathVariable Long loanId) {
        return loanService.findLoanById(loanId).map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND));
    }

    @PutMapping("/{loanId}")
    public ResponseEntity<Loan> update(@PathVariable Long loanId, @RequestBody Loan loan) {
        if (!Objects.equals(loanId, loan.getId())) throw new
RuntimeException(HttpStatus.BAD_REQUEST);
        Loan stored = loanService.findLoanById(loanId)
            .orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND));
        if (stored.isReturned()) throw new RuntimeException(HttpStatus.CONFLICT); //
check AFTER 404
        return ResponseEntity.ok(loanService.saveLoan(loan));
    }

    @DeleteMapping("/{loanId}")
    public ResponseEntity<Void> delete(@PathVariable Long loanId) {
        loanService.deleteLoan(loanId);
        return ResponseEntity.noContent().build();
    }
}

```

Order of checks in `PUT` matters: **400 (shape of the request)** → **404 (does it exist)** → **409 (state)**. A missing loan must be 404, never 409. Query parameter for a filter, path variable for identity.

P3 — Black-box: `StreamQualitySelector`

A player picks a stream quality from `plan` ("Basic" or "Premium") and `bandwidthKbps`. Basic: 1000–3000 → SD, 3001–8000 → HD. Premium: 1000–6000 → HD, 6001–8000 → UHD. `bandwidthKbps` ≤ 999 or ≥ 8001 → unavailable. Any other plan → unavailable. Tasks: (1) classes, (2) minimal representative test cases, one invalid input at a time, (3) three-point boundary for **Basic, SD/HD limit**.

Answer (executed by `StreamQualitySelectorTest`)

<code>plan</code>	PC1 Valid Basic PC2 Valid Premium PC3 Invalid any other value, "", null				
<code>bandwidth</code>	BC1 Invalid ≤ 999				
	BC2 Valid Basic	1000..3000 (SD)	BC3 Valid Basic	3001..8000 (HD)	
	BC4 Valid Premium	1000..6000 (HD)	BC5 Valid Premium	6001..8000 (UHD)	
	BC6 Invalid ≥ 8001				
PT1 Basic	500	unavailable	(BC1)	PT5 Premium	7000 UHD (BC5)
PT2 Basic	2000	SD	(BC2)	PT6 Premium	9000 unavailable (BC6)
PT3 Basic	5000	HD	(BC3)	PT7 Gold	4000 unavailable (PC3)
PT4 Premium	4000	HD	(BC4)		
Boundary, Basic, last SD value L = 3000: 2999 SD 3000 SD 3001 HD					

Trap to notice: Basic 3000 → SD but Premium 3000 → HD. The same number sits in different classes depending on the other parameter — which is why the classes are split **per plan**.

P4 — Deployment diagram: smart greenhouse

1. The **Soil Sensor** is a device that runs the *Moisture Firmware* artifact. It communicates with the **Greenhouse Controller** over **Zigbee**.
2. The Greenhouse Controller is a **machine** that deploys the component **Irrigation Planner**, which must request *soil readings* from the Sensor Gateway and request *weather forecasts* from the Cloud Service.
3. The **Weather Cloud** is a machine that deploys the component **Forecast Service** and communicates with the controller over **HTTPS**.
4. The **Sensor Gateway** is a component deployed on the Greenhouse Controller (Planner requires one interface from it). Draw it. Do not add anything else.

Answer: «device» Soil Sensor (with «artifact» Moisture Firmware inside) —**Zigbee**— «machine» Greenhouse Controller (components Irrigation Planner, Sensor Gateway) —**HTTPS**— «machine» Weather Cloud (component Forecast Service). Interfaces: **Soil Readings Service** (lollipop on Sensor Gateway, socket on Irrigation Planner — both *inside the same node*, no line between nodes), **Weather Forecast Service** (lollipop on Forecast Service, socket on Irrigation Planner, crossing the HTTPS connection). Checklist: 3-D boxes · «machine» not «node» · artifact nested · protocol labels, no arrowheads · named interfaces · nothing extra.

P5 — Coverage and complexity

```
1 int classify(int x) {
2     if (x < 0) {
3         return -1;
4     }
5     if (x == 0) {
6         return 0;
7     }
8     return 1;
9 }
```

Tests: `classify(-5)` and `classify(7)`. (a) Line coverage? (b) Cyclomatic complexity? (c) Which one test makes branch coverage complete?

Answer: executable lines are 2, 3, 5, 6, 8 = 5. `-5` runs 2, 3. `7` runs 2, 5, 8. Union = 2, 3, 5, 8 = **4/5 = 80 %** (line 6 never runs). (b) two decisions + 1 = **3**. (c) `classify(0)` — takes the `x == 0` true branch (and gives 100 % lines).

P6 — Quiz (answers in brackets)

1. `git commit` with nothing staged and a modified file — what happens? **[Nothing is committed; the change stays *modified* and must be added first.]**
2. A test reads `if (mode == FAST) assertEquals(2, r); else assertEquals(5, r);`. Best fix? **[Split into two @Test methods, one per branch.]**
3. A test calls `assumeTrue(false)`. Result? **[Skipped — neither passed nor failed.]**
4. Which diagram shows machines, the software on them and the protocols between machines? **[Deployment diagram.]**
5. Lollipop or labelled line: how are "TLS" and "User Accounts" drawn? **[TLS = plain labelled line between nodes; "User Accounts" = lollipop/socket between components.]**
6. A feature is deployed for 5 % of users first. **[Canary release.]**
7. Two environments, traffic switched all at once. **[Blue-green deployment.]**
8. Hidden behind a flag, turned on later without redeploying. **[Feature toggle.]**
9. "The app shall load within 2 s on 4G" — type? **[Non-functional (performance efficiency).]**
10. INVEST: "As a user I want the system to be fast." Which letter fails? **[Testable (and Estimable).]**
11. Which pipeline stage runs first in CI? **[Build / compile, then static analysis — cheap, fast checks first (fail fast).]**
12. A unit test needs a fixed answer from a collaborator; nothing is verified. **[Stub.]** The test must verify `pay()` was called once. **[Mock.]**
13. `assertEquals(0.1 + 0.2, 0.3)` fails. Fix? **[Add a delta: `assertEquals(0.3, 0.1 + 0.2, 0.0001)`.]**
14. Which can a full statement-coverage suite still miss? **[A decision's false branch — branch coverage is stronger.]**
15. Mistake → ? → ? → Failure. **[Fault/defect → Error.]**
16. Continuous Delivery vs Deployment? **[Delivery needs a manual approval for production; Deployment is fully automatic.]**
17. Which relationship: part dies with the whole? **[Composition (filled diamond at the whole).]**
18. Which principle: add behaviour without editing existing classes? **[Open-Closed Principle.]**
19. `git fetch` vs `git pull`? **[fetch downloads only; pull = fetch + merge.]**
20. Tool that scans every pull request for null-pointer risks before building? **[Automated static analysis.]**